j'ai un nom de domaine items.ch, j'aimerais créer une page html statique dans mon hébergment github.com/jmparatte, comment configurer mon dns items.ch et quels fichiers supplémentaires je devrais ajouter au futur répertoire github?

Pour utiliser ton domaine **items.ch** avec une **page HTML statique hébergée sur GitHub**, tu peux utiliser **GitHub Pages**. Voici la configuration complète : DNS + fichiers à ajouter.

---

## 1️⃣ Créer le dépôt GitHub Pages

Dans ton compte GitHub (GitHub), crée un dépôt.

Deux possibilités :

### Option simple (recommandée)

Nom du repo :

```
items.ch
```

ou par exemple :

```
items
```

Puis active **GitHub Pages** dans :

```
Settings → Pages
```

Choisis :

```
Deploy from branch → main → /root
```

---

## 2️⃣ Fichiers à ajouter dans le repo

Dans ton dépôt, crée au minimum :

```
/index.html
```

Exemple :

```html
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>items.ch</title>
</head>
<body>
  <h1>Bienvenue sur items.ch</h1>
</body>
</html>
```

---

### Fichier important : `CNAME`

Ajoute aussi un fichier **CNAME** (sans extension) :

```
CNAME
```

contenu :

```
items.ch
```

Cela indique à GitHub que ce repo utilise ce domaine.

---

## 3 Configuration DNS de items.ch

Dans ton DNS chez ton registrar (Infomaniak / Gandi / etc), ajoute :

### Pour le domaine racine

```
items.ch     A     185.199.108.153
items.ch     A     185.199.109.153
items.ch     A     185.199.110.153
items.ch     A     185.199.111.153
```

Ces IP sont celles de **GitHub Pages**.

## Optionnel : www

```
www.items.ch     CNAME     jmparatte.github.io
```

## 4️⃣ Activer HTTPS

Dans GitHub :

```
Settings → Pages
```

activer :

```
Enforce HTTPS
```

GitHub génèrera automatiquement un certificat via **Let's Encrypt**.

---

## 5️⃣ Structure finale du repo

```
items.ch/
 ├ index.html
 ├ CNAME
 └ éventuellement
    ├ style.css
    ├ script.js
    └ images/
```

---

## 6️⃣ URL finale

Après propagation DNS (~5-30 min):

```
https://items.ch
```

servira ton `index.html` .

---

### ✅ Avantage de cette méthode

- gratuit
- CDN GitHub
- HTTPS automatique
- aucune maintenance serveur

---

💡 Si tu veux, je peux aussi te montrer :

- **la configuration DNS exacte pour Infomaniak**
- **comment faire un site ultra minimal (1 fichier HTML de 10 lignes)**
- **comment ajouter un formulaire ou analytics sans serveur**.

bien. je souhaite un sous-domaine test.items.ch, montre-moi la configuration dns infomaniak et un site ultra minimal, et comment ajouter un formulaire

Voici une configuration **simple et propre** pour :

- `items.ch` → site principal
- `test.items.ch` → site de test
- hébergement statique via **GitHub Pages**
- DNS chez **Infomaniak**

---

# 1️⃣ Configuration DNS chez Infomaniak

Dans **Manager → Domaines → items.ch → DNS**

Ajoute :

## Domaine principal

```
Type    Nom     Cible
A       @       185.199.108.153
A       @       185.199.109.153
A       @       185.199.110.153
A       @       185.199.111.153
```

---

## Sous-domaine test

```
Type    Nom     Cible
CNAME   test    jmparatte.github.io
```

GitHub redirigera ensuite vers le bon repository.

---

## 2️⃣ Organisation GitHub recommandée

Dans ton compte **jmparatte** :

### repo production

```
items.ch
```

### repo test

```
test.items.ch
```

---

## 3️⃣ Fichiers à mettre dans chaque repo

### repo `items.ch`

```
index.html
CNAME
```

### CNAME

```
items.ch
```

---

### repo `test.items.ch`

```
index.html
CNAME
```

### CNAME

```
test.items.ch
```

---

## 4 Site HTML ultra minimal

`index.html`

HTML

```html
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>items.ch</title>
</head>

<body>
<h1>items.ch</h1>

<p>Site minimal.</p>

</body>
</html>
```

C'est **le minimum absolu** pour une page valide.

---

## 5 Ajouter un formulaire sans serveur

Un site GitHub Pages ne peut **pas exécuter de backend**, mais on peut utiliser un service externe.

Une solution simple :

**Formspree**

---

### exemple de formulaire

HTML

```html
<h2>Contact</h2>

<form action="https://formspree.io/f/TON_ID" method="POST">

<label>Email</label><br>
<input type="email" name="email" required>

<br><br>

<label>Message</label><br>
<textarea name="message" required></textarea>

<br><br>

<button type="submit">Envoyer</button>

</form>
```

Fonctionnement :

1. quelqu'un remplit le formulaire
2. Formspree reçoit la requête
3. tu reçois un email

Aucun serveur nécessaire.

---

## 6 Version ultra minimaliste avec formulaire

HTML

```html
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>items.ch</title>
</head>

<body>

<h1>items.ch</h1>

<form action="https://formspree.io/f/TON_ID" method="POST">
<input type="email" name="email" placeholder="email">
<br><br>
<textarea name="message" placeholder="message"></textarea>
<br><br>
<button>envoyer</button>
</form>

</body>
</html>
```

---

## 7️⃣ Activer HTTPS

Dans GitHub :

```
Settings
Pages
Enforce HTTPS
```

Le certificat sera généré via **Let's Encrypt**.

---

✅ Résultat final :

```
https://items.ch
https://test.items.ch
```

hébergés gratuitement.

---

💡 Si tu veux, je peux aussi te montrer :

- une **architecture GitHub propre pour gérer prod/test facilement**

- un **déploiement automatique via git push**
- ou un **formulaire qui enregistre les messages dans Google Sheets** (très pratique pour un site statique).

attention, je ne souhaite ni modifier le site web items.ch, ni www.items.ch

Parfait — si **items.ch** et www.items.ch ↗ pointent déjà vers un site existant, il ne faut **rien modifier pour ces deux entrées DNS**. On ajoute **uniquement le sous-domaine** `test.items.ch` .

---

## 1️⃣ Configuration DNS chez Infomaniak

Dans :

```
Manager → Domaines → items.ch → DNS
```

ajoute simplement :

```
Type      Nom     Cible
CNAME     test    jmparatte.github.io
```

Cela signifie :

```
test.items.ch → jmparatte.github.io
```

et **ne touche pas** à :

```
items.ch
www.items.ch
```

---

## 2️⃣ Dépôt GitHub

Dans ton compte **GitHub** `jmparatte` , crée un repo :

```
test.items.ch
```

---

## 3 Fichiers du repo

Structure minimale :

```
test.items.ch/
 ├ index.html
 └ CNAME
```

---

### fichier `CNAME`

```
test.items.ch
```

---

## 4 HTML ultra minimal

`index.html`

```html
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>test.items.ch</title>
</head>

<body>

<h1>Test items</h1>
<p>site de test</p>

</body>
</html>
```

---

## 5 Activation GitHub Pages

Dans le repo :

```
Settings
Pages
```

choisir :

```
Deploy from branch
main
/root
```

Ensuite renseigner :

```
Custom domain
test.items.ch
```

GitHub générera le certificat HTTPS via **Let's Encrypt**.

---

## 6️⃣ Résultat final

Tu auras :

```
site existant      → https://items.ch
site existant      → https://www.items.ch
site GitHub test   → https://test.items.ch
```

sans impact sur la production.

---

✅ Cette architecture est **la plus sûre** pour tester avec GitHub Pages.

---

💡 Si tu veux, je peux aussi te montrer un **truc très utile pour GitHub Pages** :
comment créer automatiquement :

```
test.items.ch/dev
test.items.ch/v1
test.items.ch/demo
```

sans créer plusieurs repos (pratique pour tests rapides).

montre-moi un formulaire qui enregistre les messages dans Google Sheets

La méthode la plus simple pour un **site statique (GitHub Pages)** consiste à utiliser **Google Sheets** + **Google Apps Script** comme backend léger.
Le formulaire HTML envoie les données à un script web qui écrit directement dans la feuille.

Je te montre une configuration **simple et robuste**.

---

## 1 Créer la feuille Google Sheets

Dans **Google Sheets** crée une feuille :

```
items-contact
```

Première ligne :

```
date | email | message
```

---

## 2 Créer le script

Dans la feuille :

```
Extensions
Apps Script
```

Remplace le code par :

```JavaScript
```

```
function doPost(e) {

  var sheet = SpreadsheetApp.getActiveSpreadsheet().getActiveSheet();

  var data = [
    new Date(),
    e.parameter.email,
    e.parameter.message
  ];

  sheet.appendRow(data);

  return ContentService
    .createTextOutput(JSON.stringify({"result":"success"}))
    .setMimeType(ContentService.MimeType.JSON);
}
```

---

## 3 Déployer comme Web App

Dans **Google Apps Script** :

```
Deploy
New deployment
```

Type :

```
Web app
```

Paramètres :

```
Execute as: Me
Who has access: Anyone
```

Tu obtiendras une URL :

```
https://script.google.com/macros/s/XXXXXXXX/exec
```

---

## 4 Formulaire HTML pour ton site GitHub

Dans ton `index.html` :

HTML

```html
<h2>Contact</h2>

<form id="contact-form">

<input type="email" name="email" placeholder="email" required>
<br><br>

<textarea name="message" placeholder="message" required></textarea>
<br><br>

<button type="submit">Envoyer</button>

</form>

<p id="status"></p>

<script>

const scriptURL = "https://script.google.com/macros/s/XXXXXXXX/exec";
const form = document.getElementById("contact-form");

form.addEventListener("submit", e => {

  e.preventDefault();

  fetch(scriptURL, {
    method: "POST",
    body: new FormData(form)
  })
  .then(response => {
    document.getElementById("status").innerHTML="Message envoyé";
    form.reset();
  })
  .catch(error => {
    document.getElementById("status").innerHTML="Erreur";
  });

});

</script>
```

---

## 5 Résultat

Quand quelqu'un envoie :

| date | email | message |
| --- | --- | --- |
| 2026-03-07 | user@email.com ↗ | hello |

est automatiquement ajouté dans **Google Sheets**.

---

## 6️⃣ Avantages

✅ fonctionne sur **GitHub Pages**
✅ gratuit
✅ pas de serveur
✅ données faciles à exploiter
✅ export CSV possible

---

## 7️⃣ Améliorations utiles

Tu peux facilement ajouter :

```
nom
sujet
ip
user-agent
```

ou envoyer **un email automatique** via Apps Script.

---

💡 Si tu veux, je peux aussi te montrer une **version encore plus propre pour sites statiques** :

- **anti-spam intégré**
- **captcha invisible**
- **stockage + email automatique**

(ça tient dans **30 lignes de code** et c'est très fiable pour GitHub Pages).

Is this conversation helpful so far?